



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2233 — PROGRAMACIÓN AVANZADA 2026-1

Examen

30 de Junio 2026

Verdadero/False [X % nota final]

Distribución de puntaje (máx. 9 puntos)

- (1 punto) F: El código se ejecuta desde la carpeta anterior a “`codigo/`”, por lo que “`archivo.txt`” se escribe en la carpeta anterior.
- (1 punto) F: Para ejecutar un tests en específico es necesario indicar la carpeta, el módulo (archivo `.py`), el nombre de la clase y el nombre del test.
- (1 punto) V: El revertir los cambios o realizar cambios en una copia del tablero, asegura que las siguientes iteraciones del algoritmo pasen por todos los posibles estados del tablero.
- (1 punto) V: Se puede implementar un Itablero sin un Iterador implementado el método `__getitem__()` o retornando una estructura de datos que ya tiene implementando su propio Iterable/Iterador.
- (1 punto) F: Para poder calcular el promedio necesitamos poder calcular la cantidad y la suma de las notas, por lo que se necesita una copia del generador o una EDD que permita almacenar esa información en cada iteración del `reduce`.
- (1 punto) V: Se puede realizar lo pedido utilizando `re.split(r'[;|!]', texto)`.
- (1 punto) Dependiendo del enfoque que se le de a esta pregunta respecto a si el grafo implementado es un Grafo con o sin pesos, la respuesta puede ser Verdadero o Falso:
 - [Grafo con pesos] V: Dado que los nodos no tiene un orden definido, es posible sustituir las listas de la lista de adyacencia por *sets*.
 - [Grafo sin pesos] F: Se necesita de una 3ra estructura (diccionario, lista, tupla) para relacionar tanto la conexión a un nodo como el peso de dicha conexión.
- (1 punto) V: Una Lista Ligada es un tipo de Grafo, por lo que se puede recorrer utilizando ambos algoritmos.
- (1 punto) F: Un `array` es una estructura de datos que almacena una colección de elementos, todos del mismo tipo de dato.

Desarrollo

Pregunta 1. Modelación [Y % nota final]

1. Modelación

1.1. ■ **(1 punto)** Debe tener las siguientes clases en el lado del servidor (con cualquier nombre):

- Servidor: Orquesta las etapas del juego y contiene el resto de las partes
- Estado: Tiene almacenado el estado de cada jugador, y de los bonificadores.
- ClienteHandler: Tiene la conexión con el cliente y recibe la información de este.
- TickHandler: Envía la información del servidor a todos los clientes.

■ **(1.25 punto)** Debe tener las siguientes clases en el lado del cliente (con cualquier nombre):

- Listener: recibe los mensajes del servidor
- Sender: envía el estado del jugador al servidor cada milisegundo
- JuegoLogic: maneja la logica central del juego y la carrera.
- Kart: Controla a un personaje en el juego (el jugador y los competidores)
- Ventana: Clase de PyQt que muestra la ventana principal del juego

■ Solucion coherente (0.25 puntos)

Estas son las clases principales, es probable que el alumno tenga más clases, pero estas son las que no pueden faltar. Tanto en el servidor como en el cliente dar 0.25 por cada clase que este y sea correctamente explicada.

1.2. **(1 punto)** Todas las clases mencionadas excepto por Kart, Servidor y Estado deben funcionar en su propio thread. Quitar 0.25 por cada clase mencionada que no sea su propio thread.

1.3. **(1 punto)** ¿Cómo se comunican las clases mencionadas? Especifica las señales de PyQt, cualquier comunicación por la red y eventos de Threads.

■ **(0.5 puntos)** Networking: Los senders del cliente y servidor envían mensajes de web a los listeners del lado opuesto. Listener escucha a tickhandler, sender envía a ClienteHandler.

■ **(0.5 puntos)** Señales de PyQt: Kart, JuegoLogic y Ventana se comunican a través de señales de PyQt.

2. (2 puntos)

```
1 def esperar_conexiones(socket):
2     hacer_puerto_disponible()
3
4     # 1 punto: esperar en un loop hasta tener 8 clientes
5     # (for i in range(8) esta bien tambien)
6     while len(self.clientes) < 8:
7         socket_cliente = esperar_cliente_nuevo(socket)
8
9         # 1 punto: crear un thread que maneja el cliente
10        # para poder esperar a otro cliente
11        nuevo_handler = ClienteHandler(socket_cliente)
```

```

12     self.clientes.append(nuevo_handler)
13     nuevo_handler.start()

```

3. Bonificadores de velocidad:

- 3.1. ■ **(0.2 puntos)** La clase Listener recibe el mensaje a través del socket y envía una señal a JuegoLogic para que controle la lógica del bonificador en el juego local.
- **(0.4 puntos)** JuegoLogic envía una señal a Ventana para que muestre lo que se deba mostrar
 - **(0.4 puntos)** Cuando el jugador activa la tecla correspondiente al bonificador, le envía una señal a JuegoLogic
 - **(0.4 puntos)** JuegoLogic utiliza a Sender para hacer request del bonificador al servidor
 - **(0.4 puntos)** Listener recibe el mensaje del servidor y envía una señal a JuegoLogic para avisar del bonificador
 - **(0.2 puntos)** JuegoLogic actualiza la velocidad del Kart del jugador y utiliza un thread o un qtimer para esperar que pasen 10 segundos y actualizar la velocidad de vuelta

3.2. usando lock

```

1 def solicitar_bonificador(id_jugador):
2     # 1 punto: usar un lock para el contador
3     with self.lock_contador:
4         if self.bonificadores_disponibles > 0
5             self.bonificadores_disponibles -= 1
6             return True
7     # 1 punto: indicar de alguna manera cuando la
8     # solicitud de bonificadores falla porque no queda
9     return False

```

usando queue

```

1 mensajes = queue.Queue()
2 ...
3 while True:
4     mensaje = mensajes.get()
5     ....
6     if type(mensaje) == "bonificador":
7         if self.bonificadores_disponibles > 0
8             self.bonificadores_disponibles -= 1
9             responder(mensaje, true)
10            responder(mensaje, false)
11            ....
12            mensajes.task_done()

```

4. **(1.5 punto)** UDP para actualizar el estado del juego ya que se hace cada 1 milisegundo. Si se pierde un mensaje no afecta la logica del juego, solo hay lag. (1 punto) También se podría complementar usando TCP para el proceso de los bonificadores de tal manera de no perder información en caso de que un jugador solicite un bonificador. (0.5 puntos)

Pregunta 2. Serialización [25 % nota final]

Distribución de puntaje (máx. 6 puntos)

A. Diseño del protocolo (1 punto)

La siguiente es una posible solución. Se codifican `username` y `accion` en UTF-8, mientras que `datos` se serializa con JSON y luego se codifica en UTF-8.

El mensaje se organiza de la siguiente manera:

Encabezado			Contenido		
L_u	L_a	L_d	<code>username</code>	<code>accion</code>	<code>datos</code>
4 bytes	4 bytes	4 bytes	L_u bytes	L_a bytes	L_d bytes

Cada largo se representa como un entero de 4 bytes.

- (0.5 puntos) El encabezado permite identificar los límites del mensaje o de cada campo.
- (0.5 puntos) Explica el protocolo propuesto.

Se acepta cualquier protocolo consistente que transforme toda la información a *bytes* y permita reconstruirla.

B. Función serializar (2.5 puntos)

Una posible implementación, recibiendo los tres campos por separado, es:

```
1 import json
2
3
4 def serializar(username, accion, datos):
5     username = username.encode("utf-8")
6     accion = accion.encode("utf-8")
7     datos = json.dumps(datos).encode("utf-8")
8
9     encabezado = (
10         len(username).to_bytes(4, byteorder="big")
11         + len(accion).to_bytes(4, byteorder="big")
12         + len(datos).to_bytes(4, byteorder="big")
13     )
14
15     return encabezado + username + accion + datos
```

Distribución de puntaje:

- (0.5 puntos) Incluye y serializa toda la información, independientemente de cómo la reciba la función.
- (0.5 puntos) Obtiene el contenido como *bytes*. Debe usar `encode` si su serialización produce un `str`; no es necesario si ya produce *bytes*.
- (0.5 puntos) Calcula los largos sobre los *bytes* y construye el encabezado.
- (0.5 puntos) Construye el mensaje según el protocolo propuesto.
- (0.5 puntos) Retorna una secuencia de *bytes* y es consistente con la parte A.

También es válido que la función reciba un único diccionario:

```
1 def serializar(solicitud):
2     ...
3
4
5 solicitud = {
6     "username": username,
7     "accion": accion,
8     "datos": datos,
9 }
```

No se descuenta puntaje por la cantidad o forma de los parámetros, siempre que se serialicen los tres campos solicitados.

C. **Función deserializar (2.5 puntos)** Una posible implementación es:

```
1 def deserializar(mensaje):
2     largo_username = int.from_bytes(mensaje[0:4], byteorder="big")
3     largo_accion = int.from_bytes(mensaje[4:8], byteorder="big")
4     largo_datos = int.from_bytes(mensaje[8:12], byteorder="big")
5
6     fin_username = 12 + largo_username
7     fin_accion = fin_username + largo_accion
8     fin_datos = fin_accion + largo_datos
9
10    username = mensaje[12:fin_username].decode("utf-8")
11    accion = mensaje[fin_username:fin_accion].decode("utf-8")
12    datos = json.loads(mensaje[fin_accion:fin_datos].decode("utf-8"))
13
14    return {
15        "username": username,
16        "accion": accion,
17        "datos": datos,
18    }
```

Distribución de puntaje:

- **(0.5 puntos)** Lee correctamente el encabezado.
- **(0.5 puntos)** Extrae el contenido usando los largos.
- **(0.5 puntos)** Revierte la codificación y serialización utilizadas. Debe usar `decode` solo si anteriormente utilizó `encode`.
- **(0.5 puntos)** Recupera correctamente los tres campos.
- **(0.5 puntos)** Retorna la información y es consistente con las partes A y B.

Nota: también es válido reunir los tres campos en un único diccionario y serializarlo completo con JSON o pickle. En ese caso, el encabezado puede contener solamente el largo total. Lo importante es que el mensaje sea una secuencia de *bytes* y que `deserializar` revierta correctamente lo realizado por `serializar`. Por ejemplo, `json.dumps` produce un `str` y requiere `encode`; en cambio, `pickle.dumps` produce directamente *bytes*.